Binary Fish School Search applied to Feature Selection

João André Gonçalves Sargo
Joao.sargo@ist.utl.pt

Instituto Superior Técnico, Lisboa, Portugal

Outubro 2013

Abstract

The aim of the present paper is to develop efficient feature selection approaches. A novel wrapper methodology for feature selection is formulated based on the Fish School Search (FSS) optimization algorithm, intended to cope with premature convergence. In order to use this population based optimization algorithm in feature selection problems, the use of binary encoding for the internal mechanisms of the fish school search is proposed, emerging the binary fish school search (BFSS). The proposed algorithm, as well as other state of the art feature selection methods such as Sequential Forward Selection (SFS) and Binary Particle Swarm Optimization (BPSO), were combined with fuzzy modelling in a wrapper approach and tested over two databases, a benchmark and an ICU (intensive care unit) database. The purpose of using this last database was to predict the readmission of ICU patients 24 to 72 hours after being discharged. Several statistical measures were considered to characterise the patient stay, including the Shannon entropy and the weighted mean. The results obtained by comparing the performance measures and the number of features selected of the used algorithms, show promising results for the novel algorithm BFSS.

Keywords: Feature Selection, binary encoding, Fish School Search, ICU, readmissions

1. Introduction

The information age is very hard to grasp. In an average person's life nowadays, we get more information in a day than someone who lived 100 years ago would get in a lifetime. The speed at which information is increasing means that finding accurate data is becoming more important than the data itself [13].

With the urgent need for a new generation of computation techniques and tools to assist humans in extracting useful information (knowledge) from a fast growing volume of data, a methodology was created, the Knowledge Data Discovery (KDD), first introduced by Fayyad in 1996 [4].

The KDD process comprises a series of steps to extract knowledge from data: 1) *data acquisition*: The process of acquiring and storing data, 2) *data preprocessing*: consists of applying proper techniques that allow the improvement of the overall quality of the data, includes processing of noise/outliers, correction of missing values, and/or alignment of data sampled at different frequencies, 3) *feature selection:* consists of finding useful features (variables) to represent the data and discarding the non-relevant ones, containing redundant information, 3) *modelling:* refers to the process of combining methods from computational intelligence and/or statistics to extract patterns in data sets. In this work it was used classification models (fuzzy modelling), which identifying to which of a set of categories (classes) a new observation belongs, on the basis of a training set of data containing observations whose category membership is known, and finally 4)*interpretation*: the process of evaluating the discovered knowledge with respect to its validity, usefulness, novelty, and simplicity. External expertise may be required in this step.

In real-world systems, the selection of a low number of features that consistently describe the problem is usually time consuming and, in many cases, impossible to achieve with a greedy approach. In this paper, the focus is turned to the Feature Selection stage of the KDD method.

The field of feature selection has been object of extensive research in recent years [12]. This is explained due to the potential benefits introduced when reducing data dimensionality. It can greatly improve data visualization and understanding, facilitating knowledge discovery. Feature selection algorithms can be grouped into four categories: filters, wrappers (used in this work), hybrids and embedded [14, 8]. Wrappers require one predetermined mining algorithm and use its performance as the evaluation criterion. They search for the features best suit to improve the performance of the mining algorithms, but they also tend to be more computationally expensive than filters [16]. Nevertheless, wrapper methods have the associated problem of having to train a classifier for each tested feature subset. This means testing all the possible combinations of features will be virtually impossible. To solve this problem, the novel binary fish school search algorithm have been proposed, a search heuristic with binary encoding, based on the Fish School Search (FSS)

optimization algorithm [7], that is able to find fairly good solutions without searching the entire workspace.

In section 2, the main concepts of fuzzy modelling are introduced. Then, the state of the art wrapper methods are presented, section 3. In section 4, the original fish school search algorithm encoded with a real number scheme is briefly explained, together with the first approach to adapt this continuous input algorithm to be used in feature selection problems. Section 5 presents the detailed formulation of the novel algorithm: binary fish school search. Section 6 presents the databases using to test the algorithms. Furthermore, in section 7 the studied wrapper methods are validated in a benchmark database and then applied to database of health care, predicting the readmission of ICU patients 24 to 72 hours after being discharged. Finally, the conclusions of this work are discussed in section 8.

2. Fuzzy Modeling

In this work it was used the machine learning technique: Fuzzy modelling. It is a tool that allows approximation of nonlinear systems when there is little or no previous knowledge of the problem to be modeled [15, 3]. This tool supports the development of models around human reasoning (also referred to as approximate reasoning), and allows an element to belong to a set to a degree, indicating the certainty (or uncertainty) of its membership.

First order Takagi-Sugeno (TS) fuzzy models [15] were applied, which consist of fuzzy rules where each rule describes a local input-output relation. When first order TS fuzzy systems are used, each discriminant function consists of rules of the type:

$$R_j: \textbf{If } \mathrm{x_1} \text{ is } \mathrm{A_{j1}} \text{ and } \ldots \text{ and } \mathrm{x_N} \text{ is } \mathrm{A_{jN}} \quad \textbf{then } \mathrm{y_j} = (\mathrm{a_j})^{\mathrm{T}}\,\mathrm{x} + \mathrm{b_j} \tag{1}$$

where, $\mathrm{j} = 1, \ldots, \mathrm{J}$ corresponds to the rule number, $\mathrm{x} = (\mathrm{x1}, \ldots, \mathrm{xN})$ is the input vector, N is the total number of inputs (features), $\mathrm{A_{jn}}$ is the fuzzy set for rule Rj and $\mathrm{n^{th}}$ feature, and $\mathrm{y_j}$ is the consequent function for rule Rj. The degree of activation of the $\mathrm{j^{th}}$ rule is given by:

$$\beta_{\mathrm{j}} = \prod_{\mathrm{n=1}}^{\mathrm{N}} \mu_{\mathrm{A_{jn}}}(\mathrm{x}) \tag{2}$$

where $\mu_{\mathrm{Ajn}}(\mathrm{x}) : \mathrm{R} \rightarrow [0, 1]$. The number of rules j and the antecedent fuzzy sets $\mathrm{A_{jn}}$ were determined by means of fuzzy clustering in the product space of the input and output variables. In this work, the chosen clustering method was fuzzy C-means (FCM), [1], since it demonstrated to be the most suitable for classification.

Given a classification problem, and being a linear consequent, a threshold t is required to turn the continuous output $\mathrm{y} \in [0, 1]$ into the binary output $\mathrm{y} \in \{0, 1\}$. In this way, if $\mathrm{y} < \mathrm{t}$ then $\mathrm{y} = 0$, and if $\mathrm{y} \geq \mathrm{t}$ then $\mathrm{y} = 1$.

3. Wrapper Methods

The main characteristic of wrapper methodologies is the involvement of the predictor as part of the selection procedure. In this work, a learning machine was used as a "black box" to score the subsets according to their predictive performance [8].

Wrappers are constituted by three main components:

1) Search method;
2) Learning machine;
3) Feature evaluation criteria.

Wrapper approaches were aimed to improve the results of the specific predictors they work with. During the search, subsets were evaluated without incorporating knowledge about the specific structure of the classification [8]. In this work it was used two search algorithms, the sequential forward selection (SFS) and the metaheuristic optimization algorithm binary particle swarm optimization (BPSO).

Sequential Forward Selection

A detailed description of the sequential forward selection search algorithm used is reported in [17]. Briefly, a model is built for each of the features in consideration, and evaluated using a performance criterion upon the test set. The feature that returns the best value of the performance criterion is the one selected. Then, other feature candidates are added to the previous best model, one at a time, and evaluated. Again, the combination of features that maximizes the performance criterion is selected. When this second stage finishes, the model has two features. This procedure is repeated until the stop criterion is achieved. In the end, all the relevant features for the considered process should be obtained.

The main advantages of this method relate to its simplicity, possibility of graphical representation of the performance of the added feature and transparent interpretation of the results which, for clinicians, is particular attractive. The main disadvantage is related to the greedy and thus susceptible approach of finding local optima [11].

Binary Particle Swarm Optimization

Particle swarm optimization is a stochastic population-based metaheuristic, inspired in swarming behaviour of some biological species (e.g. bird flocks). A particle is associated to a position and a velocity in the search space, where the method for determining the changes in velocity depends on the particle itself and the other particles. The iterative process in search of the optimum is [6]:

Step 1: Evaluate each particle in the swarm;
Step 2: Find the swarm and particle best values;
Step 3: Update velocities;
Step 4: Update positions of the particles;
Step 5: Go to Step 1 if not finished/stop criteria.

There are two crucial steps in the way the algorithm operates, the update of velocities and update of particle positions. This method can be queried in [6].

4. Fish School Search

The novel Binary Fish school search algorithm, formulated and presented in this work, was created based on the optimization search algorithm: Fish school search (FSS), invented by C. Bastos Filho and F. Lima Neto in 2007, [7].

Several oceanic fish species, as well as other animals, present social behaviour. This phenomenon's main purpose is to increase mutual survivability and may be viewed in two ways: (i) for mutual protection and (ii) for synergistic

achievement of other collective tasks. Here, protection means reducing the chances of being caught by predators; and synergy, refers to an active mean of achieving collective goals such as finding food. Although a few abstractions, the main characteristics derived from real fish schools and incorporated into the core of the approach are sound, and are grouped into two observable categories of behaviours as follows, emerging the operators:

Feeding – food is a metaphor for indicating to the fish the regions of the aquarium that are likely to be good spots for the search process;

Swimming – a collection of operators that are responsible for guiding the search effort globally towards subspaces of the aquarium that were collectively sensed by all individual fish as more promising with regard to the search process.

The concept of food was considered as related to the function to be optimized in the process. For example, in a minimization problem the amount of food in a region would be inversely proportional to the function evaluation in this region. The "aquarium" is defined by the delimited region in the search space where the fish can be positioned, each fish represents a possible solution. The two operators can now be described.

The Feeding Operator

As in real situations, the fish of FSS are attracted to food scattered in the aquarium in various concentrations. In order to find greater amounts of food, the fish in the school can move independently (see individual movements in the next section). As a result, each fish is allowed to grow in weight, depending on its success or failure in obtaining food. The authors proposed that fish's weight variation be proportional to the normalized difference between the evaluation of fitness function of previous and current fish position with regard to food concentration of these spots:

$$W(t+1) = W_i(t) + \frac{f[x_i(t+1)] - f[x_i(t)]}{\max\{|f[x_i(t+1)] - f[x_i(t)]|\}} \quad (3)$$

where $W_i(t)$ was the weight of the fish i, $x_i(t)$ the position of the fish i and $f[x_i(t)]$evaluated the fitness function (i.e. amount of food) in $x_i(t)$. A few additional measures were included to ensure rapid convergence toward rich areas of the aquarium, namely:

- Fish weight variation is evaluated once at every FSS cycle;
- An additional parameter, named weight scale (W_{scale}) was created to limit the weight of a fish. The fish weight may vary between 1 and W_{scale}.
- All the fish are born with weight equal to $\frac{W_{scale}}{2}$.

The Swimming Operators

For fish, swimming is directly related to all the important individual and collective behaviours such as feeding, breeding, escaping from predators, moving to more liveable regions of the aquarium or, simply being gregarious. This panoply of motivations to swim away inspired the authors of [7] to group causes of swimming into three classes: a) individual, b) collective-instinct and c) collective volition.

Below further explanations on how computations are performed on each of them are provided.

a) Individual Movement

Individual movement occurs for each fish in the aquarium at every cycle of the FSS algorithm. The swim direction is randomly chosen. Provided the candidate destination point lies within the aquarium boundaries, the fish assess whether the food density there seems to be better than at its current location. If not, or if the step-size would be considered not possible (i.e. lying outside the aquarium or blocked by, say, reefs), the individual movement of the fish would not occur. Soon after each individual movement, feeding would occur, as detailed above.

For this movement, a parameter was defined to determine the fish displacement in the aquarium called individual step ($step_{ind}$). Each fish moves $step_{ind}$ if the new position has more food than the previous position. Actually, to include more randomness in the search process the individual step is multiplied by a random number generated by a uniform distribution in the interval$[-1,1]$, represented as u in (4). In this simulation, the individual step was decreased linearly in order to provide exploitation abilities in later iterations:

$$\vec{x}(t+1) = \vec{x}(t) + u(-1,1)S_{ind}(t), \quad (4)$$
$$S_{ind}(t) = step_{ind,initial} - (step_{ind,initial} - step_{ind,final})\frac{g_{atual}}{g_{final}}$$

where g_{atual} is the number of the actual iteration and $g_{final}$is the total number of iterations.

b) Collective-Instinctive Movement

After the individual movement, a weighted average of individual movement based on the instantaneous success of all fish of the school is computed. This means that fish that had successful individual movements influence the resulting direction of movement more than the unsuccessful ones. When the overall direction is computed, each fish is repositioned. This movement is based on the fitness evaluation enhancement achieved, as shown in (5):

$$x_i(t+1) = x_i(t) + I(t), \quad (5)$$
$$I(t) = \frac{\sum_{i=1}^{N} \Delta x_{indi}\{f[x_i(t+1)] - f[x_i(t)]\}}{\sum_{i=1}^{N}\{f[x_i(t+1)] - f[x_i(t)]\}}$$

where Δx_{indi} is the displacement of the fish i due to the individual movement in the FSS cycle.

c) Collective-Volative Movement

After individual and collective-instinctive movements are performed, one additional positional adjustment is still necessary for all fish in the school: the collective-volitive movement. This movement is devised as an overall success/failure evaluation based on the incremental weight variation of the whole fish school. In other words, this last movement will be based on the overall performance of the fish school in the iteration. The rationale is as follows: if the fish school is putting on weight (meaning the search has been successful), the radius of the school should contract; if not, it should dilate. This operator is deemed to help greatly in enhancing the exploration abilities in FSS. This phenomenon might also occur in real swarms, but the reasons are as yet unknown. The fish-school dilation or contraction is applied as a small step drift to every fish

position with regard to the school's barycenter. The fish-school's barycenter is obtained by considering all fish positions and their weights, as shown in (6):

$$Bari(t) = \frac{\sum_{i=1}^{N} x_i(t) W_i(t)}{\sum_{i=1}^{N} W_i(t)} \quad (6)$$

For this movement, a parameter called volitive step (step_{vol}) was defined as well. The new position is evaluated as in (7) if the overall weight of the school increases in the FSS cycle; if the overall weight decreases, (8) should be used.

$$x_i(t+1) = x_i(t) - step_{vol}.rand.[x_i(t) - Bari(t)] \quad (7)$$

$$x_i(t+1) = x_i(t) + step_{vol}.rand.[x_i(t) - Bari(t)] \quad (8)$$

where rand is a random number uniformly generated in the interval [0,1]. We also decreased the linear step_{vol} along the iterations.

The FSS algorithm starts by randomly generating a fish school according to parameters that control fish sizes and their initial positions.

Regarding dynamic, the central idea of FSS is that all bio-inspired operators perform independently from each other. The FSS search process is enclosed in a loop, where invocations of the previously presented operators will occur until at least one stop condition is met. Stop conditions conceived for FSS are as follows: limitation of the number of cycles, time limit, maximum school radius and maximum school weight. Examples of the use of FSS algorithm can be visualized in [7].

Decimal to Binary Fish School Search

The FSS algorithm, described above, is a high dimension search optimization process in continuous space. The first intuitive and logical approach was to use the FSS continuous algorithm to search a one dimension integer number that would then be transformed in a vector of binary input in the objective function. To do so, the decimal to binary system representation was chosen. Inspired by [6], in a problem with N_t variables, a state is encoded by a sequence of N_t bits, each bit indicating whether a feature is present or absent. An example of a possible state is represented by the sequence:

$$\boldsymbol{xi} = (x_{i1}, x_{i2}, \ldots, x_{iN_t}) = (1, 0, \ldots, 1) \quad (9)$$

This relatively simple representation allows the original algorithm, to select features in the without major changes in the original algorithm. All the operators were used as describe above. The only modification needed was to round the position of each fish to its nearest integer. Thus, it was only necessary to use one dimension on the search space to search the decimal system representation of the solution. This approach was called the Decimal to Binary Fish School search (D2BFSS) algorithm. In order to evaluate the fitness of the decimal system solution (position of the fish), the integer solution was transformed to its binary representation, which was a vector of 0 or 1 bits with dimension equal to the maximum number of features to be selected as represented in (9). The following fitness function was used to describe the performance of the selected features during the FS process:

$$f = \alpha(1 - P) + (1 - \alpha)\left(1 - \frac{N_f}{N_t}\right) \quad (10)$$

where N_f represents the number of features selected and N_t the total number of features while the value P accounts for the performance of the model created. Constant $\alpha \in [0, 1]$ is the weight of the related goal: accuracy or subset size. Recalling that the two main objectives in the FS problem are: maximizing the model accuracy and minimizing the size of the feature subset.

5. Binary Fish School Search

It is important to note that the D2BFSS approach, presented above, does not manipulate the vector of bits (feature selected) in its internal mechanisms (the FSS movement operators). Thereby, the decimal to binary system approach may have problems with convergence, low performance or even be a random search.

With this concern in mind, it was decided to modify the internal mechanisms of the FSS algorithm to manipulate binary inputs himself. The following sections describe the modifications to the fish school search algorithm, emerging the binary fish school search.

Encoding

There are various ways of encoding a problem solution, the encoding presented here was inspired in [6]. An example of a possible state (position of a fish) can be represented by the sequence used in objective function of the D2BFSS approach, equation (9). This binary scheme, offers a straightforward representation of a feature subset, allowing the algorithm to search through the workspace, adding or removing features, simple by flipping bits in the sequence. While the FSS algorithm was not originally developed in the context of binary encoding, it appeared to be possible to modify the real to a binary encoding, keeping the following principles:

- to follow the internal mechanisms of the original algorithm, without losing the meaning of each operator;
- to add few additional parameters;
- to ensure the convergence of the algorithm;
- to keep simplicity and understanding to the modifications.

In the next sections, the modifications made to each of FSS internal mechanisms are presented.

a) Initialization of each fish

For each fish i, the initial position was initialized randomly by doing:

$$x_{ij} \leftarrow \begin{cases} 1, & if\ r > 0.5 \\ 0, & otherwise \end{cases} \quad i = 1, \ldots, N, j = 1, \ldots, N_t \quad (11)$$

where r is a random number uniformly generated in the interval $[0,1]$, N the number of fishes and N_t the total number of features to be selected.

By doing this, the algorithm starts with completely random positions, being the number of features selected at

the start around Nt/2. If the initial number was too small, the algorithm might not converge freely along iterations.

b) Individual Movement

The Individual movement occurs once in every cycle of the BFSS. For each fish i, and for each bit j, if a random number k (uniform distribution in the interval [0,1]), is smaller than Sind(t) the bit will flip, otherwise it will not change:

$$x_{ij} \leftarrow \begin{cases} \overline{x_{ij}}, & if\ k < S_{ind}(t) \\ x_{ij}, & otherwise \end{cases} \quad i = 1, \dots, N,\ j = 1, \dots, N_t \qquad (12)$$

Parameter Sind, in the same way as the FSS, will decrease linearly along the iterations depending on the first value and the last value of stepind. This allows a soft convergence through the iterations. A fish will move if the new position has more food than the previous position, i.e. if the fitness function of new set of features selected (new position) has a better performance than the previous one. By doing this, the random exploration of each individual fish is preserved.

b) Collective-Instinctive Movement

After the individual movement, the weighted average of the individual movements, based on fishes that had moved, is calculated. This process was executed in the same way as the FSS, equation (3).

In order to make all fishes head to the direction of the successful individual movement position some changes had to be made to the original FSS algorithm. When dealing with positions with bits), equation (5) loses its meaning. The displacement of the fish, Δx_{indi} in equation (5), can no longer be quantified correctly using the discrete flipping of a bit.

For that reason, equation (13) was used to describe the resultant position of the overall successful of the individual movement:

$$\vec{I}(t) = \frac{\sum_{i=1}^{N} x_{ind} \Delta f_i}{\sum_{i=1}^{N} \Delta f_i} \qquad (13)$$

In (13), Δx_{indi} in (5) was replaced with x_{ind}. In this approach, the use of the actual position of the fishes that had success in the individual movement is seen as being more descriptive than the flipping of bits.

The resulting vector $\vec{I}$ has the same dimension as the positions of the fishes, but with values varying between 0 and 1. As an illustrative example, (14) represents a possible configuration of $\vec{I}$:

$$\vec{I} = [0.1 \quad 0.5 \quad 0 \quad 0.3 \quad \dots \quad 0.7] \qquad (14)$$

The goal of the Collective-Instinctive Movement operator is to attract each fish to the resultant direction of the individual movement operator. In the Binary Fish School Search each fish approaches $\vec{I}$. To do so, it is necessary for it to have bit format. After some preliminary tests, the use of an adaptive threshold per iteration was chosen. For a given $\vec{I}(t)$, a threshold was used, multiplying the parameter thres_c by the max value of $\vec{I}(t)$. The resultant value of this multiplication would then be used as threshold in the current iteration for this operator: if the values of the bits of $\vec{I}(t)$ were below the threshold, they would be considered 0, otherwise 1.

For the example (14), if the parameter trhe_c was 0.4, the threshold used in this iteration would be $0.4 * 0.7 = 0.28$, considering 0.7 the max value of (14), resulting in:

$$\vec{I} = [0 \quad 1 \quad 0 \quad 1 \quad \dots \quad 1] \qquad (15)$$

Therefore, for each iteration t, the threshold to compute $\vec{I}(t)$ binary vector was calculated using the max value of $\vec{I}(t)$. This allowed the algorithm to select at least 1 feature, less likely to lose its exploration ability in comparison of the case using a constant threshold through all iterations.

After the computation of $\vec{I}(t)$ in bit format, all fish position could now tend to $\vec{I}(t)$. To do so, the position of each fish was compared with $\vec{I}(t)$. One bit (randomly chosen) of the fish that did not have the same value as $\vec{I}(t)$ was flipped. This process approaches the position of each fish to $\vec{I}(t)$. In comparison with the original algorithm, $\vec{I}(t)$ no longer represents the direction but the position resultant of the successfully individual movements. By only flipping one bit per fish, a soft and steady convergence of the algorithm is expected. An illustrative example can be represented:

$$x(t) = [0\ 1\ 0\ 1\ 1] \rightarrow I = [0\ 1\ 0\ 0\ 0] \qquad (16)$$
$$x(t+1) = [0\ 1\ 0\ 0\ 1]$$

In (16) the fish x(t) moved in the direction of $\vec{I}(t)$. The bits with the same values of $\vec{I}(t)$ are represented in red. The resultant position, $x(t+1)$, is achieved by flipping one random bit that has different values, represented in green. The total number of bits in red in the new position is greater than the one in the position before the collective-instinctive movement, making the new position of the fish to be closer to $\vec{I}(t)$.

b) Collective-Volative Movement

Similarly to the Collective-Instinctive Movement operator, the Collective-volitive operator underwent some changes. The main goal of this operator is, depending of the success of the individual movement, to contract or dilate the fish position to or from the barycentre.

The barycentre was computed in the same way as in the FSS algorithm (6). Analogously to the computation of the vector $\vec{I}(t)$, after (6) the barycentre was not obtained in a bit format. Thereby, the parameter tresh_v was introduced. In the same way as in the collective-instinctive movement, the adaptative threshold was used: multiplying tresh_v with the max value of barycentre.

If the overall individual movement was a success (overall weights improved in the iteration) each fish would approximate to the barycentre. Similarly to the process in the collective-instinctive movement operator, every bit per fish were compared to the barycentre. One bit (chosen randomly) that was not the same value as the barycentre was then flipped. By making only one flip per fish, the algorithm enables a soft directing from the previous position to the new one, closer to the barycentre. An illustrative example to the case of the improvement of the overall weights (contraction) is shown in (17).

$$x(t) = [0\ 1\ 0\ 1\ 1] \rightarrow bari = [0\ 1\ 0\ 0\ 0] \qquad (17)$$
$$x(t+1) = [0\ 1\ 0\ 0\ 1]$$

In (17), fish x(t) changed randomly one of its bits that were different (green) from barycentre (bari). This allowed the fish to approximate to the baricenter.

If the overall weights had not improved, each fish has to move to the opposite direction of the barycentre. To do this, the concept of anti-barycenter is introduced, consisting of a vector with the same dimensions as the barycentre but with flipped bits. In this situation, the process is the same as described above for the case of contraction to the barycentre but using the anti-barycentre. In (18) the representation of the case of not improvement of the overall weights of the example (17) is presented:

$$x(t) = [0\ 1\ 0\ 1\ 1] \rightarrow antibari = [1\ 0\ 1\ 1\ 1] \quad (18)$$

$$x(t+1) = [1\ 1\ 0\ 1\ 1]$$

In (18), the fish new position x(t + 1) is obtained comparing each bit with the anti-baricenter of the barycentre presented in (17). One of the bits with different values (green), was flipped, making the new position of the fish to be closer to the antibari and consequently further to bari in (17). After the collective-volitive movement, a new cycle begins.

Although some of the parameters of the BFSS algorithm influence the final number of features selected (use of thresholds), the process of developing an objective function is critical, since it serves as guidance in search of the optimum. The objective function in (10) was used to evaluate the fitness of a certain position (fish). The spot criterion used was the number of iterations, and the best solution per iteration was the fish with best performance in the fitness function for that iteration.

When performing the modifications presented above, some parameters were introduced. Fig. 1 summarises the set of parameters used in the FSS, D2BFSS and BFSS algorithms.

FSS	D2BFSS	BFSS
•No. of fishes	•No. of fishes	•No.of fishes
•No. of iterations	•No. of iterations	•No. of iterations
•W_{scale}	•W_{scale}	•W_{scale}
•$step_{ind}$ [inital and final]	•$step_{ind}$ [initial and final]	•$step_{ind}$ [initial and final]
•$step_{vol}$ [inital and final]	•$step_{vol}$ [initial and final]	•**thres_c**
	•**α**	•**thres_v**
		•**α**

Fig. 1 Parameters for the original FSS, the decimal to binary FSS and the binary FSS

6. Data

Two databases were considered, the sonar benchmark database [19, and the four readmission datasets from the MIMIC II [20] database, with the purpose of predicting the readmission of patients in an intensive care unit (ICU) during 24 to 72 hours after being discharged [5].

The sonar database consists of 208 samples with 60 features and two classes.

The four readmission datasets considered, were created based on a developed dataset result of the work [5] where the raw data of the MIMIC II was treated. In the present work, after the preprocessing of the dataset from [5] four different readmission datasets were created. For each of the four datasets the mean, the standard deviation, the maximum, the minimum, the weighted mean and the Shannon entropy [18, 2] in order to describe the time series of the physiological variables [5] were considered. However, for each one of the four readmission datasets a different gradient for linear distribution of the weights for the weighted mean were considered: 0.1, 0.4, 0.6 and 0.9. Thus, giving rise the four different readmission databases. Each dataset was formed with 726 patients with 132 features and two classes: readmitted or not readmitted.

Traditionally, accuracy has been used to evaluate classifier performance. This measure is defined as the total number of correct classifications over the total number of available samples. Usually, most of the classification problems have two classes, positive and negative cases [10]. Thus, the classified test points can be divided into four categories:

- True positives (TP)- correctly classified positive cases,
- True negatives (TN) - correctly classified negative cases,
- False positives (FP)- incorrectly classified negative cases,
- False negatives (FN)- incorrectly classified positive cases.

Given these categories, the accuracy can be written as:

$$ACC = \frac{no.of\ true\ positives + true\ negatives}{no.of\ true\ positives + false\ positives + false\ negatives + true\ negatives} \quad (19)$$

This criterion is limited, especially in medical applications, for various reasons. If one of the classes is more underrepresented than the others, misclassifications in this class will not have a great impact in the accuracy value. Also, a good classification of a class might be more important than classifying other classes and this cannot be assessed with accuracy. To take this matter into account, two performance measures were introduced:

$$sensitivity = \frac{number\ of\ true\ positives}{number\ of\ true\ positives + false\ negatives} \quad (20)$$

$$specificity = \frac{number\ of\ true\ negatives}{false\ positives + true\ negatives} \quad (21)$$

The sensitivity and specificity varies between [0,1].

The receiver operating characteristic (ROC), or simply ROC curve, is a graphical plot which illustrates the performance of a binary classifier system as its discrimination threshold is varied. It is created by plotting the fraction of true positives out of the positives (sensitivity) vs. the fraction of false positives out of the negatives (one minus the specificity), at various threshold settings. When using normalized units, the area under the curve (AUC) is equal to the probability of a classifier ranking a randomly chosen positive instance being higher than a randomly chosen negative one (assuming 'positive' ranks higher than 'negative'). The AUC measure ranges from 0.5 (random classifier) to 1 (perfect classifier).

In the present work, the Sensitivity and Specificity were used as performance measures for the models using the sonar database and readmission datasets. However, in the models using sonar database accuracy was used as the main performance measure, and, in the readmission datasets the AUC. This choice was made because of the fact that the two classes of the sonar database had similar numbers of samples (89 samples 45% vs 111 samples 55%) and for the readmission problem the percentage of the

class readmitted was only 12.3% against 87.7% of not readmitted. In this case one of the classes is underrepresented and if the accuracy had been used the results would not be realistic

7. Results

The main objective of this section is to evaluate the applicability of the proposed search optimization algorithms. These methods combine the machine learning algorithm, in section 2, with the state of the art search algorithms presented in section 2, 3 and 4 and are used in the databases announced in 6. They will be compared with each other and with the results obtained without FS, based on the predictive performance and the number of selected features.

The use of a learning machine in wrapper methods, so as to evaluate subset suitability, involves a correct feature subset assessment. The process described next, was preformed for each of the databases considered in this work.

The data was firstly divided in two groups, the feature selection (FS) subset and the model assessment (MA) subset. This division was random but with the same percentage of each class in each subset, i.e. 50% of the samples belong to the FS and the other 50% to the MA subset, and both groups had the same percentage of samples for each class considered.

The FS subset was divided in 70% of the samples for training set and 30% for testing set. This division was also performed randomly and with the same percentage of each class in each set. The feature selection was then accomplished and, after the stop criterion was reached, the model with the best performance was selected. The features selected, as well as its threshold, were then recorded and a 10-fold cross validation was performed to the MA subset.

The k-fold cross validation consists of dividing data into k subsets, using at each time $k-1$ for training and the other set for testing. Each subset had been divided to have the same percentage of samples for each class. For k times, models were trained and tested using the recorded features and threshold, until all possible combinations of training and testing sets were covered. The results are the average of the performance measure, introduced in section 2.2.3, on all splits. The mean and the standard deviation of the AUC, sensitivity and specificity and accuracy are then reported. The k-fold cross validation allows the evaluation of the validity and robustness of the discovered model by assessing how the resulting model from feature selection would generalize to an independent data set (MA subset).

To reduce variability, introduced by the division of the data not only in the FS/MA subset but also in the division of train/test subsets, 10 rounds of the 10-fold cross validation process were performed always using different partitions. The round with the best performance was selected, and the performance measures of that round described the model created with the selected set of features.

Sonar database

The D2BFSS and BFSS algorithms presented were not tested before, so, in order to find an appropriate set of parameters, a widely study of the parameters were preformed. For the sonar database, the selected parameters for the D2BFSS where: $step_{ind}$=[1e18 1e13]([initial and final values]), $step_{vol}$[1e18 1e5], W_{scale}=50 and α=0.1 . For the BFSS the set was: thres_c=0.7, thres_v=0.3, $step_{ind}$=[0.01 0.001], W_{scale}=100000 and α=0.5. The parameters for the BPSO algorithm were used as indicated in [6]. The stop criterion for these algorithms was 300 iterations in the FS process. The results of the comparison of the different FS algorithms using the sonar database are presented in Table 1, 100 rounds of the process were performed always using different partitions of the data for the FS/MA and train/test subsets. However the same partitions were used between the different algorithms in order to do a fare comparison of the results.

The main measures of performance here compared were the mean accuracy (%) of the 10 fold crossvalidation process and the number of features selected.

As expected, the FS algorithms using metaheuristics optimization algorithms (D2BFSS, BFSS and BPSO) achieved better results in comparison with the SFS and no feature selection approaches. Regarding the number of features selected, the D2BFSS method had lower performance in comparison with the BFSS and the BPSO. This, and the fact that the D2BFSS presented a poor convergence throughout the iterations in the FSS process (see Fig. 2), led to the conclusion that the use of the decimal to binary system was not the best approach to achieve results similar to the state of the art algorithms, reinforcing the need to modify the internal mechanisms of the FSS algorithm to deal with binary input, the BFSS.

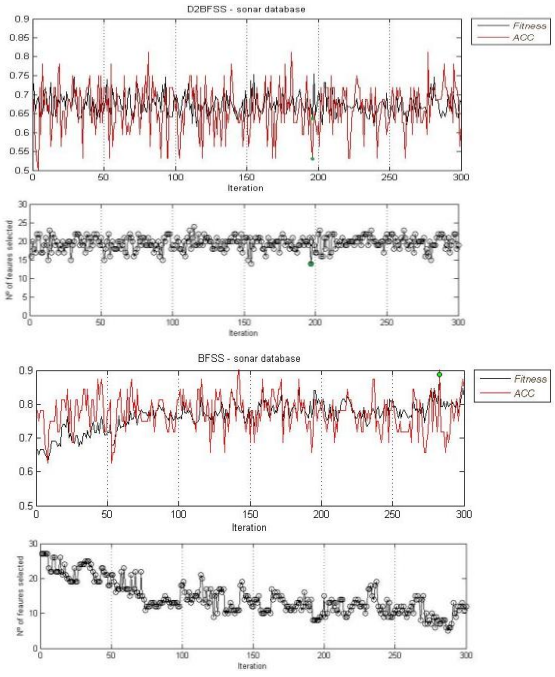

Fig. 2 Graphical evolution of the FS best model per iteration, for the D2BFSS (above) and the BFSS (below) algorithms.

Table 1- Comparison between the studied FS approaches using sonar database.

		10-fold cross validation									
		mean				standard deviation					
	100 rounds	AUC	ACC %	sensitivity	specificity	AUC	ACC %	sensitivity	specificity	threshold	features selected
NO FS	mean	0,74	74,66	0,77	0,71	0,11	11,51	0,17	0,19	0,48	60
	std	0,03	3,59	0,054	0,05	0,02	2,53	0,04	0,05	0,041	0
	best round	0,83	83,57	0,87	0,79	0,1	9,61	0,08	0,15	0,5	60
SFS	mean	0.73	73.19	0.77	0.69	0.12	12.05	0.18	0.21	0.48	8
	std	0.04	4.14	0.07	0.08	0.03	2.59	0.05	0.05	0.05	4
	best round	0.81	81.51	0.87	0.75	0.15	15.13	0.13	0.24	0.45	9
D2BFSS	mean	0.74	74.59	0.78	0.71	0.13	12.67	0.18	0.2	0.48	13
	std	0.04	3.74	0.07	0.08	0.02	2.25	0.04	0.04	0.05	1
	best round	0.84	83.8	0.88	0.8	0.11	11.61	0.16	0.19	0.45	15
BFSS	mean	0.72	72.85	0.78	0.67	0.12	12	0.18	0.22	0.46	7
	std	0.04	4.27	0.08	0.1	0.02	2.14	0.05	0.05	0.05	2
	best round	**0.85**	**85.62**	**0.95**	**0.76**	**0.09**	**9.29**	**0.09**	**0.16**	**0.43**	**8**
BPSO	mean	0.73	73.3	0.78	0.67	0.13	13.2	0.17	0.2	0.47	8
	std	0.04	4.31	0.09	0.09	0.04	3.59	0.05	0.05	0.05	2
	best round	0.82	82.67	0.92	0.73	0.1	9.28	0.12	0.15	0.45	7

It can be concluded that, although the BPSO algorithm had selected 1 less feature in comparison to the BFSS, the performance of BFSS algorithm greatly exceed the BPSO. It is believed that the presence of the collective-volitive operator in the BFSS, which enables the fishes to contract or expand per iteration, is the main tool allowing the algorithm to not converge to local maxima and, thereby, obtaining better results.

Readmission database

Due to the high number of features to be selected and the high number of samples, in comparison to the sonar data base , it was chosen to perform 500 rounds of the whole process (FS+MA), always with different partitions of the data for the FS/MA and train/test sets. However, the same partitions between the different FS algorithms were used, allowing the results to be comparable. For each of the four datasets, the SFS, the BFSS and the BPSO algorithms were applied to the Feature selection. Due to its low overall performance and lack of convergence in the sonar database, the D2BFSS was discarded.

As the four datasets only differ in the features of the weighted mean, it was decided to do the study of parameters for only one dataset. It was concluded that a proper set of parameters for the BFSS using this database is: thres_c=0.1, thres_v=0.1, $step_{ind}$=[0.01 0.001], W_{scale}=500 and α=0.1. The BPSO parameters used were selected from the previous study [6] that used similar readmission databases. In exception of the parameters, α=0.5. The spot criterion was 500 iterations.

After collecting the results of all the methods of feature selection for the readmission datasets, it was concluded that the use of different gradients for the distribution of the weights in the weighted mean proved to be not relevant. All the algorithms show almost no sensitivity to the presence of different gradients of the weighted mean for the 22 physiologic variables, presenting similar results for the four datasets using the same algorithm. Thus, the results for only one dataset can be presented, Table 2.

The results of the comparison of the different features algorithms are not as obvious as the ones with the benchmark sonar database were, possibly due to the variability introduced dealing with real data in high dimensions spaces. Once again, the BFSS achieved better overall results in comparison with all methods used, achieving slightly better mean of AUC and less features selected.

It is noteworthy to mention that in all of the datasets used, achieved considerably less features than all the other methods, maintaining the convergence of the algorithm and slightly better performance of the models.

It is also relevant to note that the sensitivity of the best model using the BFSS algorithm is considerably better in comparison with that of other algorithms. Recalling that, this measure indicates the true positive rate, i.e. the patients who were selected as readmitted and were in fact.

In title of example, Fig. 3 shows the graphical evolution of the model created in the best round of the BFSS method using the dataset with gradient=0.9, showed in Table 2.

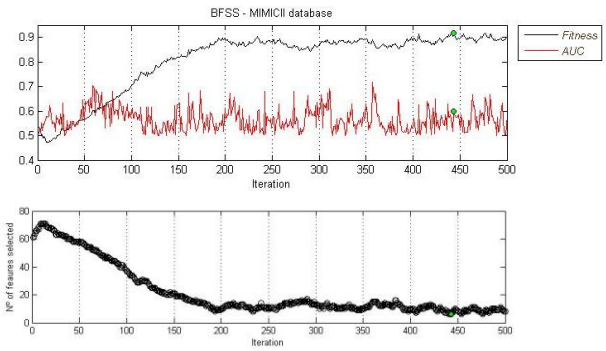

Figure 3: Graphical evolution of the BFSS process. Evolution of the fish with best performance per iteration (above) and evolution of the number of features selected of the fish with the best performance per iteration (below).

The graphical evolution in Fig 3, unquestionably confirms the convergence of the novel BFSS algorithm formulated in this thesis. In the first ~200 iterations there is a transitory dynamic, where the BFSS algorithm mainly converges to a lower number of features. In the rest of the iterations, the algorithm oscillates, searching for the optimal combination of a low number of features and a high performance of AUC.

Table 2: Comparison between the studied FS approaches using the readmission dataset with gradient: 0.9.

Gradient:0.9											
		mean, 10-fold cross validation				standard deviation, 10-fold cross validation					
	500 rounds	AUC	ACC %	sensitivity	specificity	AUC	ACC %	sensitivity	specificity	threshold	features selected
NO FS	mean	0.58	48.77	0.71	0.46	0.06	28.45	0.34	0.37	0.32	132
	std	0.02	9.80	0.11	0.13	0.02	4.82	0.06	0.06	0.05	0
	best round	**0.65**	**67.58**	**0.63**	**0.68**	**0.10**	**11.73**	**0.21**	**0.14**	**0.20**	**132**
SFS	mean	0.64	56.73	0.73	0.54	0.08	17.33	0.24	0.22	0.14	8
	std	0.02	6.14	0.08	0.08	0.02	4.15	0.06	0.05	0.02	2
	best round	**0.68**	**66.43**	**0.71**	**0.66**	**0.10**	**14.59**	**0.14**	**0.17**	**0.15**	**9**
BFSS	mean	0.61	53.69	0.71	0.51	0.07	21.35	0.28	0.28	0.13	5
	std	0.03	7.73	0.10	0.10	0.02	5.74	0.08	0.08	0.03	1
	best round	**0.69**	**55.53**	**0.87**	**0.51**	**0.11**	**14.12**	**0.11**	**0.16**	**0.15**	**6**
BPSO	mean	0.59	57.66	0.62	0.57	0.08	14.60	0.23	0.18	0.14	10
	std	0.05	9.75	0.21	0.14	0.03	6.58	0.07	0.09	0.03	3
	best round	**0.67**	**59.13**	**0.78**	**0.57**	**0.12**	**10.23**	**0.31**	**0.14**	**0.10**	**8**

8. Conclusions

This work has addressed the problem of feature selection. Firstly, the background for machine learning was presented, followed by the definition of its application to the problem of classification. The fuzzy classification modeling was briefly described. The modifications that allowed the optimization algorithm Fish School Search to be used in binary input problems were formulated and then applied to the feature selection problem, emphasizing the novel Binary Fish school Search. Other state of the art algorithms for FS were also described, namely the SFS and the BPSO approaches. Finally, the proposed methods were validated over the benchmark sonar database and then applied to a case of study: the prediction of patient's readmission in an ICU within 24-72h period following their discharge. This section summarizes the conclusions acquired during the development of this work, and suggests topics for future research.

Binary Fish School Search

In general, the modification of an existing algorithm is risky. In advance, it is very difficult to the algorithm developer to guarantee that the algorithm will have a better performance that the state of the art algorithms or even to assure convergence.

There are countless ways to modify algorithms with real encoding schemes in order to solve binary input problems. In this thesis, two novel approaches to the FSS algorithm were presented, the D2BFSS and the BFSS. The decimal to binary approach was achieved through simple passage from continuous to discrete system in the usage of the objective function to the original FSS algorithm. The BFSS was a more complex approach that modified the internal mechanisms of the FSS algorithm, allowing the procedure itself to manipulate the binary inputs.

Results of the benchmark sonar database for the feature selection problem showed that simple manipulation of the fitness function, in order to transform the objective function from decimal to binary system, achieved low performance, as well as a very low convergence evolution, when compared to the BFSS algorithm. This result was expected due to the simplicity of the D2BFSS, and reinforced the need to implement internal changes in FSS algorithm.

The results of the two databases tested showed that the initial development goals proposed for the BFSS algorithm were achieved: to remain faithful to the original internal mechanisms of the FSS without losing the meaning of each operator, addiction of few number of parameters, overall simplicity and understanding of the modifications, and, mainly, the convergence of the algorithm.

However, it is important to note that the number of parameters used in the BFSS process is high in comparison with other Feature Selection algorithms, and some limitations to the proposed algorithm can arise if there is a poor adjustment in the parameters.

To conclusion, this brand new binary optimization algorithm exceeded expectations, with not only its convergent evolution but also with good results in comparison with the other features selection algorithms, especially the BPSO. The main contribution to achieve such results is believed to have been the presence of the collective-volitive operator, which had a major role in the exploration process of the algorithm, and consequent capacity of avoiding local maxima.

It is also important to note that although the formulated BFSS algorithm was used in the problem of feature selection, it can be applied to any optimization problem with a binary encoding of the inputs, opening doors for future fields of research

Prediction of readmissions

The dimensionality of medical data is typically very high. Hence, the great importance of using algorithms of feature selection, for this environment, improving early detection of medical problems. Patients readmitted to an intensive care unit during the same hospitalization have an increased length of stay, higher costs and increased risk of death. Previous studies have demonstrated overall readmission rates of 4-14% [21, 22], of which nearly a third can be attributed to premature discharge from the critical care setting [21, 23]. It is also documented that the length of stay for readmitted patients is at least twice as long as that for patients discharged from the ICU but not readmitted and that hospital death rates are 1.5 to almost 10 times higher among ICU readmission [24].

The proposed wrapper methods are appropriated to this problem due to various reasons, such as the possibility to deal with large databases, the capacity to predict outcomes by means of a small subset of features, and the possibility to bring out new set of variables never considered before.

However, the proposal of using different gradients for the linear distribution of the weights in the weighted mean to describe the time series of the physiologic variables, proved not to be significant. It was expected that one gradient for the weights of the weighted mean would stand out, the results show that the use of different gradients did not present a significant benefit in the performance of the models nor in the number of Features Selected.

In this work, various modifications to the FSS algorithm were proposed. Nevertheless, not all the possible paths were explored. The reduction of the number of parameters to be used and the refinement of the BFSS operators are logical paths to future investigation.

Studies [9] already proved that the use of dynamic values for the weights of the fishes in FSS algorithms can be beneficial, this improvement could be also advantageous for the BFSS algorithm presented in this work.

It is also important to test the BFSS applied to feature selection in other benchmark databases with different dimensions in order to evaluate the full potential in comparison with other state of the art algorithms.

References

[1] R. Babuska. Fuzzy Modeling for Control. International Series in Intelligent Technologies. Kluwer Academic Publishers, Norwell, MA, USA, 1st edition. April 1998.

[2] J. Dayou, N. C. Han, H. C. Mun, A. H. Ahmad. Classification and Identification of Frog Sound Based on Entropy Approach. IPCBEE, vol. 3, pages 184-187. 2011.

[3] A. P. Engelbrecht. Computational Intelligence: An Introduction. New York: John Wiley. 2003.

[4] U. Fayyad, G. Piatetsky-Shapiro, P. Smyth. From data mining to knowledge discovery in databases. AI Magazine, 17(3), pages 37-54. 1996.

[5] A.S. Fialho, F. Cismondi,S.M. Vieira, S.R. Reti, J.M.C. Sousa, S.N. Finkelstein. Data mining using clinical physiology at discharge to predict ICU readmissions. Expert Systems with Applications 39. 2012.

[6] A. S. Fialho, F. Cismondi, S. M. Vieira, J. M. C. Sousa, S. R. Reti, M. D. Howell, and S. N. Finkelstein. Predicting outcomes of septic shock patients using feature selection based on soft computing techniques. In Proc. of the IPMU 13th International Conference, pages 65-74. 2010.

[7] C. J. A B. Filho. , F. B. L. Neto, A. J. C. C. Lins, A. I. S. Nascimento, M. P. Lima. A Novel Search Algorithm based on Fish School Behavior. IEEE International Conference on Systems, Man, and Cybernetics - SMC. 2008.

[8] I.Guyon, S. Gunn, M. Nikravesh, and L. A. Zadeh, editors. Feature Extraction: Foundations and Applications (Studies in Fuzziness and Soft Computing). Springer. August 2006.

[9] A. G. K. Janecek, Y. Tan. Feeding the fish - weight update strategies for the fish school search algorithm. ICSI'11 Proceedings of the Second international conference on Advances in swarm intelligence , Volume Part II. 2011.

[10] M. A. Mazurowski, P. A. Habas, J. M. Zurada, J.Y. Lo, J. A. Baker, G. D. Tourassi. Training neural network classifiers for medical decision making: The effects of imbalanced datasets on classification performance. Neural Networks, 21(2-3):427-436. 2008.

[11] L. F. Mendonça, S. M. Vieira, J. M. C. Sousa. Decision tree search methods in fuzzy modeling and classification. International Journal of Approximate Reasoning, 44(2), pages106-123. 2007.

[12] H. Liu, H. Motoda. Computational Methods of Feature Selection. Chapman and Hall. 2007.

[13] G. E. Moore. Cramming more components onto integrated circuits. Electronics Magazine, 4. 1965.

[14] Y. Saeys, I. Inza, P. Larranaga. A review of feature selection techniques in bioinformatics. Bioinformatics, 23(19), pages 2507–2517. 2007.

[15] J. M. C. Sousa, U. Kaymak. Fuzzy Decision Making in Modeling and Control. World Scientific Singapore. 2002.

[16] L. Talavera. An evaluation of filter and wrapper methods for feature selection in categorical clustering. In Advances in Intelligent Data Analysis VI, volume 3646 of Lecture Notes in Computer Science, pages 742-742. Springer Berlin / Heidelberg. 2005.

[17] R. O. Duda, P. E. Hart. Pattern Classification and Scene Analysis. Wiley & Sons. 1973.

[18] A. Partap, B. Singh. Shannon and Non-Shannon Measures of Entropy for Statistical Texture Feature Extraction in Digitized Mammograms. WCECS, vol. II. 2009.

[19] I. Guyon, A. Elisseeff. An introduction to variable and feature selection. Journal of Machine Learning Research, 3, pages 1157-1182. 2003.

[20] M. Saeed, C. Lieu, R. Mark. MIMIC II. a massive temporal icu database to support research in intelligence patient monitoring. Computers in Cardiology, 29, pages 641-644. 2002.

[21] W. Baigelman, R. Katz, G. Geary. Patient readmission to critical care unit during the same hospitalization at a community teaching hospital. Intensive Care Med, 9(5), pages 253-256. 1983.

[22] A. L. Rosenberg, C. M. Watts. Patients readmitted to intensive care units: A systematic review of risk factors and outcomes. Chest, pages 492–502, 2000.

[23] C. G. Durbin Jr, R. F. Kopel. A case control study of patients readmitted to the intensive care unit. Critic Care Med, 21, pages 1547-1553. 1993.

[24] A. L. Rosenberg, C. Watts. Patients Readmitted to ICUs: A Systematic Review of Risk Factors and Outcomes. Critical Care Reviews, 2000.